Architecture Notes

Feature Request & Voting Platform — choices, assumptions, trade-offs, limitations, and next steps.

Contents

1. Choices

CHOICE **FastAPI + SQLite + SQLAlchemy 2.0 + Alembic backend** — Python 3.12. FastAPI gives typed REST endpoints via Pydantic schemas that mirror the frontend DTOs (camelCase alias generator), SQLite is zero-config for local dev while still offering real transactions and constraints, and Alembic version-controls the schema from day one. (ADR-0004)

CHOICE **Frontend as a thin HTTP client behind a swap boundary** — the UI goes through `frontend/src/api/`, which now performs real `fetch` calls to the backend. The method surface matches the old mock exactly, so hooks, components, and pages are untouched by the mock→HTTP swap. (ADR-0002, ADR-0007)

CHOICE **JWT bearer authentication** — `POST /auth/login` verifies a bcrypt hash and issues an HS256 token; protected endpoints require `Authorization: Bearer`. The mock's mutable "role switcher" is dropped; role now comes from the token and is re-checked against the `users` table per request. Read endpoints stay anonymous. (ADR-0005)

CHOICE **Flat `{code, message}` error contract** — every domain error returns a machine-readable `code` mapped to HTTP status, matching the mock's `ApiError`. Validation failures map to `INVALID` / 400. This keeps the frontend's error handling identical across mock and real backend. (ADR-0006)

CHOICE **Support as computed `COUNT(*)`** — vote counts are derived from the `votes` table, not a stored counter, so they can never drift from the rows.

CHOICE **Merge is non-destructive and computes a union** — the survivor gains the deduplicated union of voters; the absorbed Request becomes `redirected` with `mergedInto` pointing at the survivor, remains readable, and keeps its comments/activity. (ADR-0001)

2. Assumptions

- **Reads are public.** List/detail/comments require no token; per-user fields (`votedByMe`, `canVote`) are derived when a token is present.
- **Role is authoritative server-side.** The token carries `role`, but admin endpoints re-check the stored user row; a stale/forged role claim can't escalate.
- **SQLite is a dev-grade database.** It provides real transactions and constraints (which we lean on for vote-once), but single-writer semantics and file locking are accepted for local development.
- **One process, one DB file.** Deployment is assumed single-instance against one SQLite file.

- **Seeded identities.** Users and a shared dev password come from `seed.py`; there is no registration or password-reset flow yet.
- **API contract is frozen.** DTO shapes, error codes, and business rules mirror the original mock exactly; changes are treated as contract changes.

3. Trade-offs

Topic	Chosen approach	What we gave up
Vote-once enforcement	Composite PK `(user_id, request_id)` + `INSERT ... ON CONFLICT DO NOTHING`	Relying on a SQLite-specific upsert; portable to Postgres via `ON CONFLICT` / `INSERT OR IGNORE` dialect but not identical. In exchange: idempotent, race-safe, no app-level check.
Merge semantics	Union of voters, deduplicated in-place; absorbed request preserved read-only	Union (not sum) means a user voting on both requests counts once — intended. In-place preservation over copying means comments stay on the absorbed request and un-merging stays trivial.
Support counts	`COUNT(*)` per request in the serializer	N+1 queries on list views vs. a denormalized counter. Trade-off: no drift and no counter-maintenance bug surface; cost is per-request query volume.
Database	SQLite	No server-based concurrency, no row-level locks, weaker vacuum/backup story. Trade-off: zero-config local dev with real ACID and constraints now; a clear migration path later.
Transactions	One `Session` per request; single `commit()` at end	No explicit `begin()` / `rollback()` handling or nested transactions. Simple and correct for the current operation sizes; rollback is implicit via session close on error.
ID generation	`counters` table with `UPDATE ... RETURNING`	Sequential, human-friendly IDs (`req-1`) instead of UUIDs. Safe under SQLite's write serialization; needs a lock/retry for multi-process.

4. Limitations

- LIMIT **SQLite single-writer.** Writes serialize at the DB level and readers can block a writer under the default journal mode; not suitable for multi-instance or high-concurrency production.
- LIMIT **No optimistic concurrency.** No `version` column — concurrent edits are last-write-wins and can silently overwrite each other.
- LIMIT **Merge is not concurrency-safe.** The redirected-lock check and vote reassignment are not atomic as a guard against two racing merges; the unique constraint backstops duplicate reassignments, but a race could still produce a conflicting merge outcome.
- LIMIT **Counter race under multiple processes.** `next_id` assumes single-writer; two processes could allocate the same id without row locking or retry.
- LIMIT **Insecure default `JWT_SECRET`.** Falls back to `dev-secret-change-me`; forgetting to override it in production defeats token security.

- LIMIT **No token refresh or revocation.** JWTs are valid until expiry (`JWT_EXPIRE_MINUTES`, default 1440); logout is client-side only (drop `localStorage`).
- LIMIT **No registration or password reset.** Identity comes from seed data; no account lifecycle.
- LIMIT **No rate limiting, pagination, or body-size limits.** `GET /requests` returns everything; no abuse protection.
- LIMIT **No DB-level cascade deletes.** Request deletion explicitly deletes children in the route rather than relying on `ON DELETE CASCADE`.
- LIMIT **CORS wide open.** `allow_origins=["*"]` with all methods/headers — fine for local dev, not for production.
- LIMIT **Dev dependencies bundled in runtime image.** `pytest` / `httpx` live in the same `requirements.txt` as runtime deps; the Docker image ships them.

5. Next steps

1. NEXT **Postgres migration** for server-side concurrency, row locking, and WAL. Move from SQLite upsert to Postgres `ON CONFLICT`; revisit the ID counter (or switch to sequences/UUIDs).
2. NEXT **Enable SQLite WAL mode** in the interim to improve reader/writer concurrency with zero schema change.
3. NEXT **Add optimistic concurrency** (a `version` column + `WHERE version = :expected` on edits) so concurrent edits surface a conflict instead of overwriting.
4. NEXT **Harden merge** — make the redirected-lock and reassignment atomic (single transaction with locking, or a constraint) and add a concurrency test.
5. NEXT **Secrets management** — source `JWT_SECRET` from a secret manager and fail fast if unset in production; add token refresh and server-side revocation.
6. NEXT **Account lifecycle** — registration, password reset, and email verification.
7. NEXT **Production hardening** — rate limiting, pagination on `/requests`, request-body limits, and a locked-down CORS origin allowlist.
8. NEXT **Reduce N+1** — batch `COUNT(*)` for list support/comment counts instead of per-request queries, or introduce denormalized counters with transactional maintenance.
9. NEXT **Split requirements** — separate runtime (`requirements.txt`) from dev/test (`requirements-dev.txt`) so the production image is leaner.
10. NEXT **Move DB deletes to cascade** — express child deletion as `ON DELETE CASCADE` foreign keys instead of route-level explicit deletes.

> **Reference:** decision records live in `docs/adr/0001–0007`. This document consolidates them with the current implementation.